EC8207– APPLIED BIG DATA ENGINEERING

Mini Project Assessment

FinTech Fraud Detection Pipeline

PROJECT REPORT

Pemsiri B.M.P.B	EG/2020/4110
Suraweera S.A.Y.A	EG/2020/4225
Sandamal I.M.K	EG/2020/4190

Table Of Contents

1. Introduction 3
2. Technology Stack Justification 4
   2.1 Apache Kafka 4
   2.2 Apache Spark Structured Streaming 4
   2.3 PostgreSQL 4
   2.4 Apache Airflow 4
3. Event Time vs. Processing Time 5
   3.1 The Challenge: Late Data and Out-of-Order Events 5
   3.2 Implementation: Windowing and Watermarking 5
4. Pipeline Architecture Overview 6
5. Ethics & Data Governance 7
   5.1 Privacy Implications in Financial Surveillance 7
   5.2 Data Governance Strategies 7
6. Analytical Report 9
7. Conclusion 10

1. Introduction

Due to the development of digitalization of global financial services, digital transaction fraud is increasing at a higher rate. Therefore, a real-time fraud detection system is much needed for modern financial companies to prevent money loss before transactions are finalized. Also for regulatory purposes and financial reconciliation, historical data must securely stored. To meet all those requirements, this project implement robust Lambda Architecture. This architecture able to handle massive quantities of data by using both batch and stream-processing methods. Stream processing acts as the Speed Layer to immediately evaluate and detect fraudulent transactions in real-time and batch processing acts as the Batch Layer. It is doing Extract, Transform, Load (ETL) processes to calculate and aggregate historical data and generate financial reports for the company This dual-layer approach ensures low-latency responses for immediate security threats. Also, it ensures data integrity for long-term auditing.

2. Technology Stack Justification

To build a fault-tolerant Lambda Architecture, we use industry standard tools. This architecture must be highly available. It must also be capable of processing both massive data streams and batches.

2.1 Apache Kafka

Apache Kafka is selected as the ingestion system. We use this instead of traditional message queues due to its distributed, high-throughput, and append-only log architecture. In FinTech Industry, transaction logs must be immutable and replayable to ensure auditability. Kafka act as a highly resilient buffer . It can also able to handle high-velocity transaction spikes and holds data securely and also able to protect downstream processing engines from network overloads.

2.2 Apache Spark Structured Streaming

Apache Spark chose as the main processing engine because it provides a truly unified environment. Normally , Lambda Architecture requires to maintain two entirely separate codebases.One for the Speed Layer and another for the Batch Layer. However, Spark's Structured Streaming and DataFrame API share the exact same execution engine. This helps to apply almost same queries like SQL to both real-time streams and static batches. So, it helps to reduce development overhead. Spark Structured Streaming processes incoming data in micro batches. While not strictly true event-by-event real-time, its sub-second latency is perfectly acceptable for banking systems to evaluate and block fraud. This architecture provide high data throughput and ensure exact once fault tolerance, and also ensuring no transaction is lost.

2.3 PostgreSQL

PostgreSQL is selected as the primary database for the Fraud Alerts table. It provides fast, ACID-compliant Online Transaction Processing capabilities. It allows customer service instititues to query blocked transactions. Apache Parquet is the format for the Data Warehouse to store validated safe transactions. Parquet is a columnar storage format highly optimized for Online Analytical Processing queries. By using efficient data compression, Parquet helps to reduces physical storage costs and minimizes I/O operations when reading massive datasets.

2.4 Apache Airflow

While traditional tools like cron are good for simple scripts, But for a enterprise grade data pipeline need complex dependency management. Apache Airflow selected to orchestrate the Batch Layer. It was scheduled to execute reconciliation jobs every 6 hours. Airflow is used by developers to write workflows as Python-based Directed Acyclic Graphs, providing a clear programmatic definition of task dependencies. For example, Airflow ensures data extraction completes before the PDF report generation phase begins. It provide retry mechanisms and a Web UI for monitoring the success of the Reconciliation_batch tasks, critical for financial compliance.

3. Event Time vs. Processing Time

When we talk about modern stream processing there is a difference between time that financial event actually occurred and when it arrived at the processing engine. It is a critical technical challenge, especially in distributed systems where network latency delays message delivery.

- **Event Time:** The exact time the transaction occurred at the user's device (e.g., the timestamp field inside the JSON payload: "2026-05-08 14:05:00").
- **Processing Time:** The specific time the transaction data physically arrives at the Apache Spark streaming cluster.

3.1 The Challenge: Late Data and Out-of-Order Events

If the data pipeline rely highly on Processing Time.A non-fraud transaction delayed by a network outage could be grouped into the wrong time window considering the arrival, leads to highly inaccurate fraud analysis. For example, this project implements an "Impossible Travel" fraud rule, such that if a single user transacts from two different countries within a 10 minute window, the system flags it. If the system used Processing Time, delayed safe transactions might arrive simultaneously, falsely trigger this rule and blocking a credit card.

3.2 Implementation: Windowing and Watermarking

To solve this challenge, the pipeline was made to use Event Time. Spark Structured Streaming extracts the timestamp from the JSON payload and groups the data into 10-minute sliding time windows. However, grouping strictly by Event Time introduces the critical problem of infinite state growth. Spark cannot hold data in its internal memory forever waiting for delayed data because it is reason for crash the server. This was addressed by implementing Watermarking.

The watermark implemented a strict boundary behind the maximum observed Event Time. If the latest recorded transaction time is 10:30 AM, and the watermark is set to 10 minutes, Spark holds the analytical window open in memory for late arriving data until the internal Event Time clock hits 10:40 AM. Any data that arrive with a timestamp older than the 10 minute watermark threshold is removed as too late. This complex logic ensure that Spark safely drops old window aggregates from memory, completely preventing Out-Of-Memory errors, while allows a reasonable grace period for standard network latency.

4. Pipeline Architecture Overview

- **Producer:** Python script generates simulated financial transactions continuously.
- **Kafka Message Broker:** Captures the raw transaction payloads instantly, acting as a high-velocity buffer and maintaining immutable, distributed event logs.
- **Spark Structured Streaming (Speed Layer):** Continuously consumes the transactions topic. It executes real-time anomaly detection rules (e.g., High-Value checks), evaluates "Impossible Travel" fraud over sliding 10-minute event-time windows utilizing watermarking, and pushes blocked transaction alerts directly to PostgreSQL.
- **PostgreSQL (OLTP Storage):** Sinks all critical fraud alerts . It maintains an ACID-compliant alerts table for immediate access by customer service dashboards and human review processes.
- **Spark Batch & Airflow (Batch Layer):** Every 6 hours, Airflow orchestrates the Batch processing layer. It executes Spark jobs to process historical transaction logs , separate valid transactions from anomalies, and generate comprehensive PDF reconciliation reports.
- **Parquet Data Warehouse (OLAP Storage):** Stores the historically validated transactions in columnar Parquet files.

5. Ethics & Data Governance

Deploying a real-time anomaly detection pipeline on sensitive personal financial data has significant ethical and privacy concerns. While the major goal of this pipeline is to protect users from fraudulent transactions, the system also acts as a continuous behavioral surveillance mechanism.

5.1 Privacy Implications in Financial Surveillance

The major ethical issue raised by the project is that of User Profiling. In order for the model to effectively predict any anomalies, it requires the creation of an extremely intimate baseline of regular activity from each individual user. Surveillance Creep is inevitable as the system designed for fraud detection is constantly monitoring highly sensitive information such as GPS geolocation, unique categories of purchases, rates of transactions, and large financial amounts. Such extremely sensitive data can potentially used for purposes other than those intended if those are not regulated by strict legislation.

Furthermore, there will always be instances of False Positives and Algorithmic Bias. The programmed conditions such as Impossible Travel or High-Value could lead to false positive. In case the traveler uses the application for genuine purposes and even pays for any medical emergency, the account is immediately blocked. The strict rules could also lead to discrimination of travelers with varying lifestyles and even low income individuals.

5.2 Data Governance Strategies

To secure the system from those ethical and privacy risks , strict data governance frameworks which are align with global standards such as GDPR or CCPA must applied to the pipeline's architecture.

- **Data Minimization and Masking:** This ensures the pipeline only processes data that only need for fraud detection. Personally Identifiable Information, such as exact customer names and full credit card numbers must be masked or tokenized before entering into the Kafka topics.
- **Role-Based Access Control (RBAC):** This helps to control the access to raw data streams and historical warehouses. Data Scientists training fraud models only access to anonymized datasets, Authorized personnel only able legally access the unmasked PostgreSQL database.
- **Data Retention Policies:** Financial data must not keep for long time. The Parquet Data Warehouse must be manage automated lifecycle management. The system is required to archive or cryptographically shred raw transactional data after the retention period. For financial reconciliation and yearly auditing, the system should retain only anonymous, aggregated statistics not the individual user logs.

- **Algorithmic Transparency and Human-in-the-Loop:** A correction process needs to be implemented in addition to algorithm transparency. The alerts table in PostgreSQL holds all the details of the actual mathematics equation, merchant type, and amount involved in blocking a transaction. Administrative extreme measures like suspending the entire account must not be carried out automatically through algorithms. Instead, an immediate alert is generated to a human expert, who manually reviews the case and decides on how best to act.

6. Analytical Report

FINTECH RECONCILIATION REPORT

Generated on: 2026-05-09 12:03:50

Description	Value
Total Transactions Processed	5828
Total Ingress Amount ($)	20,517,031.57
Validated Safe Amount ($)	20,278,600.42
Blocked Fraud Amount ($)	238,431.15

FIGURE 1: "FRAUD ATTEMPTS BY MERCHANT CATEGORY

Fraud Attempts by Merchant Category

Merchant Category	Fraud Count	Total Amount ($)
Retail	5	76,998.32
Groceries	4	61,474.53
Travel	6	47,474.99
Online_Subscription	3	30,860.77
Dining	2	25,557.05

FIGURE 1: "FRAUD ATTEMPTS BY MERCHANT CATEGORY

The FinTech Reconciliation shows how well the data pipeline operated and its level of effectiveness from the security point of view. It handle 5,828 transactions that included $20.5 million worth of total ingress. The system was accurate and validated $20,278,600.42 as safe, as well as isolated and blocked $238,431.15 of the fraudulent transactions.

A total of 20 instances of fraud were detected in this test, which occurred in five different merchant categories. The highest amount was blocked for the Retail merchant ($77,000), where there were 5 attacks. As far as the number of frauds in each merchant category is concerned, then we may say that the Travel Merchant had the highest number of attacks, i.e. 6 attempts amounting to $47,475.

7. Conclusion

In summary, this project was successful to demonstrate the implementation of the Lambda architecture, which can be used to analyze data at very high velocities. With the use of Apache Kafka, Apache Spark, and Apache Airflow, the system able to provides low latency fraud detection, which is needed in real-time security while at the same time performing accurate batch reconciliations, which are needed for accounting purposes. Moreover, the system proves to be resilient to any external network problems as event time processing is done by the use of watermarks. Although the technological capabilities of this distributed data pipeline are great, it is important to note that there should always be strict data governance rules when using them